AI Products & the 4-Layer Stack

Cursor & Perplexity · Month 1, Week 1, Task 3

Every AI product — no matter how different it looks to users — is built on the same four-layer stack you learned in Task 1. We will pull apart two familiar products and ask: *which component sits at which layer?* **Cursor** is an AI code editor. **Perplexity** is an AI search engine. They look completely different on the surface, but under the hood they share the same blueprint.

Cursor — AI Code Editor

A fork of VS Code where AI understands your entire codebase. It autocompletes code as you type, answers questions about your project, and can autonomously make multi-file changes from a plain English description.

Perplexity — AI Search Engine

Instead of returning a list of links, it reads the web for you and writes a direct answer with inline citations. Every answer is grounded in live web sources retrieved the moment you ask.

Both Products on the Same 4-Layer Stack

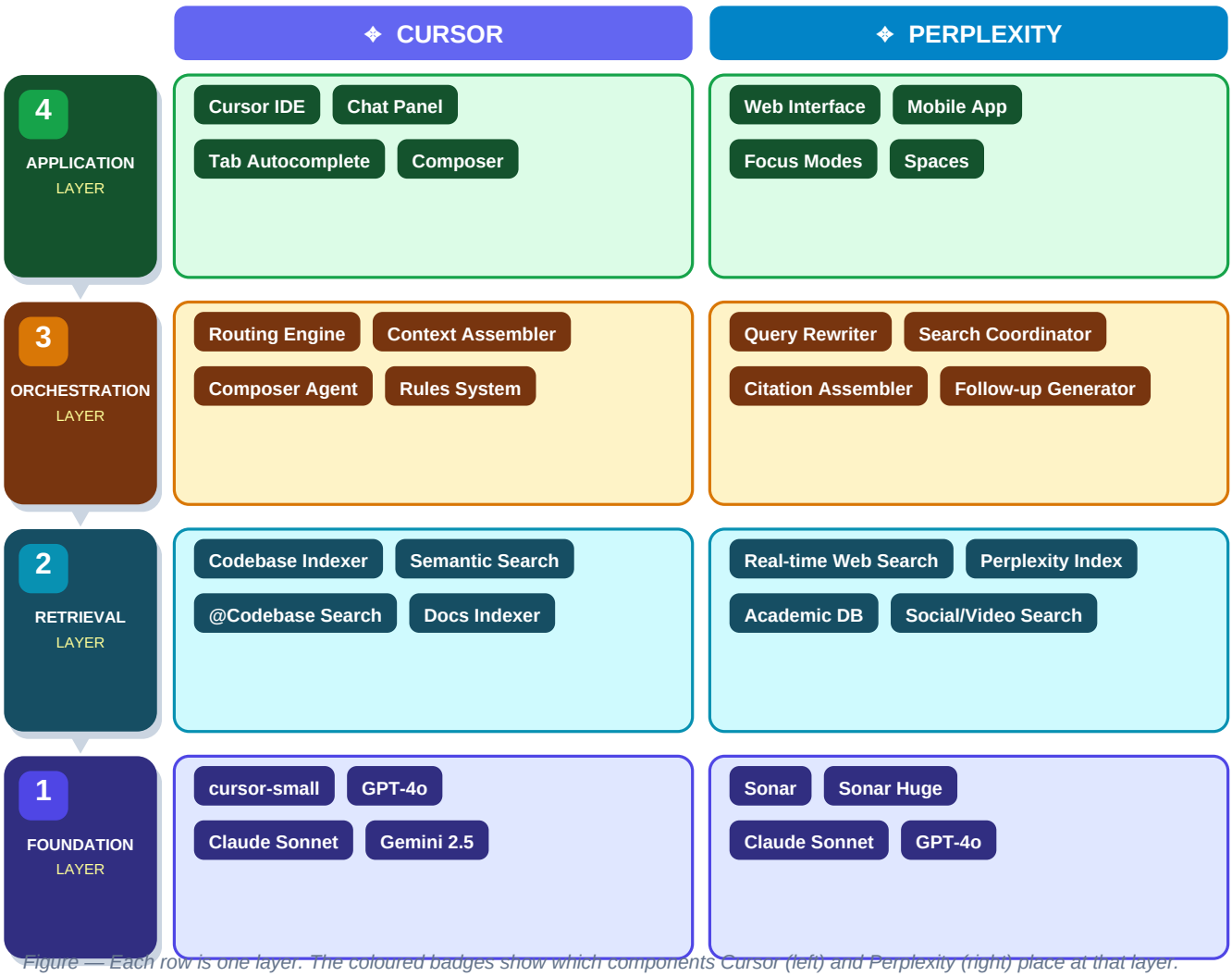

Figure — Each row is one layer. The coloured badges show which components Cursor (left) and Perplexity (right) place at that layer. Both products have components at every single layer. Pages 2–3 explain each one in full.

Cursor — AI Code Editor

VS Code fork where AI understands your entire codebase

Cursor is the clearest example of the 4-layer stack in a developer tool. Every feature a developer uses — Tab autocomplete, Chat, Composer — is powered by all four layers working together in milliseconds.

Layer 4 — Application: What developers see and touch

Component	What it does	Why this layer?
Cursor IDE	A full VS Code fork. All your extensions and shortcuts work unchanged.	*This IS the product — the surface the developer sees and touches.*
Chat Panel	Side panel for asking questions: explain this, fix this, refactor this.	*The main entry point for natural language interaction with the AI.*
Tab Autocomplete	Predicts your next lines as you type. Press Tab to accept.	*The most-used feature — surfaces AI output directly in the editor.*
Composer	Describe a change in English; Cursor plans and edits multiple files for you.	*The agentic interface — turns a natural language goal into code changes.*

Layer 3 — Orchestration: The invisible coordinator

Component	What it does	Why this layer?
Routing Engine	Picks which model to use: fast model for Tab, powerful model for Composer.	*Coordinates model selection based on task type and latency requirements.*
Context Assembler	Before each LLM call, collects your cursor position, open files, and @-mentions.	*Decides exactly what information goes into the LLM's context window.*
Composer Agent	Plans a sequence of file edits, applies them one by one, shows you a diff.	*Runs a multi-step loop: plan → act → show result → await your approval.*
Rules System	Your .cursorrules file is injected into every system prompt automatically.	*Shapes LLM behaviour project-wide without you repeating instructions.*

Layer 2 — Retrieval: Your codebase as searchable knowledge

Component	What it does	Why this layer?
Codebase Indexer	When you open a project, embeds every file into a local vector store.	*Converts your code into searchable embeddings for semantic retrieval.*
Semantic Search	Finds relevant functions and files by meaning, not just by file name.	*Retrieves the right context for each LLM call without keyword matching.*
@Codebase Search	Explicit retrieval: '@Codebase find anything about payment processing'.	*Gives you direct control over what gets retrieved and passed to the model.*

Component	What it does	Why this layer?
Docs Indexer	Embeds third-party docs (React, Next.js) so the model knows your libraries.	*Extends retrieval beyond your code to external knowledge sources.*

Layer 1 — Foundation Models: The AI brains Cursor calls

Component	What it does	Why this layer?
cursor-small	Cursor's own tiny model, built for one job: fast Tab autocomplete under 100ms.	*Speed-optimised foundation model — latency matters more than power here.*
GPT-4o	Default for most chat and standard edits. Strong general capability.	*The general-purpose brain for the majority of user-facing interactions.*
Claude Sonnet	Used for Composer and complex tasks. Excellent at large codebase context.	*Chosen when the task needs precise instruction following over many files.*
Gemini 2.5	Optional. 1M token context window for when other models run out of space.	*A specialist fallback when context length exceeds other models' limits.*

> ***Key insight:*** *When you press Tab to accept an autocomplete suggestion, all four layers fired in under 200ms. cursor-small (Layer 1) generated it. The codebase vector store (Layer 2) provided the context. The routing engine (Layer 3) chose the right model. The IDE (Layer 4) displayed the result. One keypress — four layers — all invisible to you.*

Perplexity — AI Search Engine

Real-time web retrieval + LLM synthesis with inline citations

Perplexity looks like a simple search box. Behind it is a RAG pipeline at consumer scale. The web is the database. The LLM is the synthesiser. When you study RAG in Month 2, you will build exactly what Perplexity does — just with your own documents instead of the entire web.

Layer 4 — Application: What users see

Component	What it does	Why this layer?
Web Interface	The search box, the cited answer card, and the source list on perplexity.ai.	*Everything the user sees. Hides all pipeline complexity behind a clean UI.*
Mobile App	iOS and Android apps — same functionality, optimised for smaller screens.	*A different interface surface calling the exact same underlying pipeline.*
Focus Modes	UI controls to restrict search: Web, Academic, YouTube, Reddit, or a domain.	*User-facing controls that change the retrieval strategy at Layer 2.*
Spaces	Saved research threads users can revisit, organise, and share with teams.	*Persistence built on top of the pipeline — manages search history over time.*

Layer 3 — Orchestration: The query pipeline

Component	What it does	Why this layer?
Query Rewriter	Rewrites your raw question into 2–3 better search queries before searching.	*LLM call #1 — improves retrieval quality before Layer 2 is even touched.*
Search Coordinator	Fans out to multiple sources at once based on the question type and Focus Mode.	*Decides which retrieval sources to query and merges their results.*
Citation Assembler	Maps each sentence in the final answer to the exact source URL behind it.	*Post-processes LLM output to extract and attach inline citations.*
Follow-up Generator	Generates 3–4 related questions shown at the bottom of every answer.	*A lightweight LLM call that helps users explore topics more deeply.*

Layer 2 — Retrieval: The live web as a knowledge base

Component	What it does	Why this layer?
Real-time Web Search	Queries Bing and other search APIs. Always retrieves live, fresh results.	*Primary retrieval source. Gives Perplexity knowledge no static LLM has.*
Perplexity Index	Their own web crawler and search index — not dependent on Bing alone.	*A vector store at internet scale. Their main technical competitive moat.*
Academic DB	For Academic mode: queries Semantic Scholar, arXiv, PubMed for papers.	*Specialised retrieval source for peer-reviewed, high-quality results.*

Component	What it does	Why this layer?
Social/Video Search	Queries Reddit and YouTube APIs for community opinions and video content.	*Niche retrieval sources activated by specific Focus Mode selections.*

Layer 1 — Foundation Models: Synthesising search results into answers

Component	What it does	Why this layer?
Sonar	Perplexity's own model, fine-tuned to synthesise search results into cited answers.	*Their primary model — optimised for reading retrieved pages and writing summaries.*
Sonar Huge	Their largest model, used for Deep Research mode on complex multi-step queries.	*Reserved for high-complexity tasks; trades speed and cost for maximum quality.*
Claude Sonnet	Available to Pro users for complex reasoning and long research questions.	*Third-party model offered as a premium option alongside Perplexity's own.*
GPT-4o	Available to Pro users who prefer OpenAI's model for certain query types.	*Shows that Perplexity is model-agnostic at Layer 1 — the model is swappable.*

> ***Key insight:*** *Perplexity is RAG at consumer scale. Step 1 — Query Rewriter (Layer 3) improves your question. Step 2 — Web Search (Layer 2) retrieves fresh pages. Step 3 — Sonar (Layer 1) reads those pages and writes a grounded answer. Step 4 — The web UI (Layer 4) shows the answer with citations. That is the exact same loop you will build in Month 2 of this roadmap.*

Side-by-Side Comparison

Layer	Cursor — AI Code Editor	Perplexity — AI Search Engine
4 — Application	VS Code IDE, Chat panel, Tab autocomplete, Composer agent UI	Web interface, Mobile app, Focus mode controls, Spaces
3 — Orchestration	Routing engine, Context assembler, Composer agent loop, Rules system	Query rewriter, Search coordinator, Citation assembler, Follow-up generator
2 — Retrieval	Local codebase vector store, Semantic search, @Codebase, Docs indexer	Real-time web search, Perplexity's own index, Academic DB, Social/video search
1 — Foundation	cursor-small (autocomplete), GPT-4o (chat), Claude Sonnet (Composer), Gemini 2.5	Sonar (default), Sonar Huge (deep research), Claude Sonnet, GPT-4o (Pro choice)

3 Key Takeaways

Every product has all 4 layers
Cursor and Perplexity look nothing alike to a user, yet both sit on the identical 4-layer stack. This mental model applies universally — every AI product you build or analyse fits this pattern.

The retrieval layer is what makes them different
Cursor's Layer 2 is a private vector store of your local codebase. Perplexity's Layer 2 is the live public web. Same architecture, completely different data source — this is why they solve different problems.

Layer 1 is swappable — the model is not the product
Cursor lets users switch between GPT-4o, Claude, and Gemini. Perplexity offers multiple models to Pro users. As an AI engineer, always design your product to be model-agnostic. The model at Layer 1 is an implementation detail, not the product itself.

Your Task Checklist

You have completed this task when all 5 items are done:

- ■ **Draw Cursor's stack** — On paper or Excalidraw, draw Cursor's 4-layer stack with at least 2 components per layer. Label each component with its layer number.
- ■ **Draw Perplexity's stack** — Do the same for Perplexity — draw it from memory without looking at this document. If you can draw it from memory, you understand it.
- ■ **Spot the RAG pattern** — Write one sentence: why is Perplexity a RAG system? Name the retrieval step, the generation step, and the augmentation step.

■ **Note the model diversity** — Count how many foundation models each product uses. What does it tell you that neither product is locked into a single model?

■ **Map a third product** — Pick any AI product you use — GitHub Copilot, Notion AI, Google NotebookLM. Map it to the 4-layer stack. What sits at each layer?